

Mock Design: A Ride-Hailing / Dispatch System

Phase 4 · Interview-Ready · Estimated time: 2 hours

1. Concept From First Principles: Applying the Framework

Ride-hailing (Uber/Careem/Ola-style dispatch) introduces a problem type this program hasn't covered in depth yet: **geospatial matching** at high write throughput. Millions of drivers continuously stream their GPS location, and the system must efficiently answer "which available drivers are near this rider?" within seconds — a fundamentally different access pattern than any previous mock design chapter's key-value or feed-shaped data.

Step 1 — Clarify requirements

- Functional: riders request a ride from their location; the system finds and assigns a nearby available driver; both parties track the ride's live location until completion.
- Non-functional: matching must complete within a few seconds of a request; the system must absorb continuous, extremely high-volume location updates from every active driver, not just occasional reads.

Step 2 — Estimate scale

Assume 5 million active drivers, each sending a location ping every 4 seconds, giving ~1.25 million location writes/second — a genuinely massive, continuous write load that dwarfs the ride-request read volume (perhaps a few thousand ride requests/second at peak). Stating this write:read imbalance explicitly, the mirror image of Chapter 8's read-heavy feed problem, is the single most important framing sentence for this chapter: **the design must be built around the location-update write path, not the ride-request read path.**

Step 3 — API

POST /location {driverId, lat, lng} (fired continuously by every active driver's app), POST /rides {riderId, pickupLat, pickupLng} -> {rideId, assignedDriverId}, GET /rides/{id}/location -> {lat, lng} for live ride tracking.

Step 4 — High-level design

Two logically separate services sharing one geospatial index: a **Location Service** that ingests the constant stream of driver GPS pings and updates each driver's position in an in-memory spatial index, and a **Matching Service** that, on a ride request, queries that same index for nearby available drivers and assigns one. Driver location pings flow through a message queue (Phase 2, Ch 12) to absorb the extreme write volume without blocking driver apps on synchronous writes, feeding into the Location Service's in-memory index asynchronously.

Step 5 — Deep dive: geospatial indexing and the matching algorithm

The core question: how do you efficiently find "all available drivers within 2km of this point" without scanning every driver's location on every request? A naive linear scan across 5 million drivers per ride

request is far too slow. The standard answer is **geohashing**: encode latitude/longitude into a short alphanumeric string where geographically nearby points share long common prefixes (e.g., tdr1v and tdr1w are neighboring areas). Drivers are indexed by geohash prefix (effectively grouping the map into a grid of cells), so finding nearby drivers becomes a fast lookup of drivers sharing the rider's geohash cell (and adjacent cells, to handle riders near a cell boundary), rather than a full scan. An alternative, **quadtree**-based indexing (recursively subdividing the map into smaller cells only where driver density is high), adapts better to highly uneven driver density (dense in a city center, sparse in a rural area) than geohashing's fixed-size grid cells — a real trade-off worth naming: geohashing is simpler to implement and reason about, quadtrees handle density skew more gracefully.

Matching: once nearby available drivers are found, the assignment itself isn't simply "pick the closest one" — production systems weight factors like driver rating, estimated time to reach pickup (not just straight-line distance, which can be misleading across a river or highway), and fairness (avoiding always assigning to the same driver). A strong interview answer names that a scoring function combining these factors, not raw distance alone, drives the final assignment, even without building the full scoring model live. A further subtlety worth naming: once a driver is offered a ride, they must briefly be marked unavailable (reserved, similar in spirit to Chapter 14's seat-locking pattern) so two simultaneous ride requests can't both be matched to the same driver — the exact same read-then-write race condition from Chapter 10's inventory problem, here applied to driver availability instead of stock count.

Step 6 — Trade-offs and wrap-up

Volunteer: 1.25 million writes/second against a single geospatial index is itself a scaling challenge — shard the index by geographic region (e.g., by city or by geohash prefix range), since location queries are inherently local and rarely need to span regions, making geographic sharding a natural fit rather than an arbitrary hash-based shard key.

2. Real-World Example / Case Study

Uber's own widely-discussed H3 hexagonal geospatial indexing system (a more refined evolution of the geohash idea, using hexagonal rather than rectangular cells to reduce distance-distortion at cell boundaries) is the real-world production answer to exactly this problem, and Careem, as a Uber-affiliated company, would very plausibly probe this exact geospatial indexing trade-off in a Dubai/UAE interview given its direct relevance to their actual production systems.

3. Diagram

```

Continuous driver location stream (1.25M writes/sec):
[Driver Apps] -> [Message Queue] -> [Location Service]
                                   |
                                   [In-memory Geospatial Index]
                                   (sharded by region/geohash prefix)

Ride request (matching, must be fast):
[Rider] -> [Matching Service]
          |
          query index: nearby available drivers
          (rider's geohash cell + adjacent cells)
          |
          score candidates: distance, ETA, rating, fairness
          |
          reserve chosen driver (atomic, avoids double-match)
          v
          [Assigned Driver] <-- notify --

```

4. Trade-offs Table

Decision	Alternative	Why this choice
Geohashing for spatial index	Linear scan of all driver locations	Turns $O(n)$ scan into a fast prefix-based lookup
Quadtree (density-adaptive)	Fixed-grid geohashing	Handles uneven driver density (city center vs. rural) more gracefully
Scoring function (distance+ETA+rating+fairness)	Assign closest driver by raw distance	Straight-line distance can be misleading across real road/terrain barriers
Geographic sharding of the index	Hash-based arbitrary sharding	Location queries are inherently local; geographic shards avoid cross-shard queries

5. Common Interview Questions

Q1: "How do you find nearby drivers without scanning every driver's location?"

Model approach: Name geohashing or a quadtree directly, and explain the core mechanism — grouping geographically close points so a lookup touches only a small relevant subset, not the full driver population.

Q2: "What happens if two ride requests are matched to the same driver at nearly the same instant?"

Model approach: Name the atomic reservation of driver availability directly, connecting it explicitly to the same read-then-write race condition class covered in Chapter 10's inventory problem.

Q3: "Why might you choose a quadtree over geohashing, or vice versa?"

Model approach: State the density-adaptivity trade-off directly — quadtrees handle skewed driver density better at the cost of implementation complexity; geohashing is simpler but less adaptive to dense vs. sparse regions.

6. Practice Exercises

- 1 Run the full six-step framework for "design a ride-hailing dispatch system," out loud, with a timer.
- 2 Implement a basic geohash-prefix nearby-driver lookup in pseudocode, including handling riders near a geohash cell boundary.
- 3 Design the atomic driver-reservation mechanism that prevents two simultaneous ride requests from matching the same driver.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Uber System Design Design Ride-Hailing Application System Design Interview"	Covers geospatial indexing and matching directly for this exact problem.
English	"Mock System Design Interview Design Ride Pick Up Uber OLA Lyft"	Full live mock interview format matching this chapter's structure.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual course covering geospatial and matching systems as part of the broader curriculum.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Hindi roadmap covering indexing fundamentals this chapter builds on.

8. Key Takeaways

- Ride-hailing's write load (continuous driver location pings) vastly exceeds its read load — the mirror image of Chapter 8's feed problem.
- Geohashing and quadtrees both solve the same core problem — fast nearby-point lookup — with different density-adaptivity trade-offs.
- Matching should use a scoring function (distance, ETA, rating, fairness), not raw distance alone.
- Atomic driver reservation prevents double-matching, the same race-condition class as Chapter 10's inventory problem applied to driver availability.
- A message queue absorbs the extreme location-update write volume without blocking driver apps on synchronous writes.
- Geographic sharding of the spatial index is a natural fit, since location queries are inherently local.
- Uber's real H3 hexagonal indexing system is a production-grade evolution of the geohash concept covered here.